Analytical Bias and Variance Correction for Quantized Linear and Convolutional Layers

ACIQ project (derivation, after Nagel et al. 2019)

1 Setup and Notation

We consider a single weight-bearing layer that produces pre-activation output $\mathbf{y}$ from input $\mathbf{x}$ using FP32 weights $\mathbf{W}$ and bias $\mathbf{b}$. After post-training quantization the same layer uses quantized weights $\widetilde{\mathbf{W}}$. Bias is kept in FP32. We define the per-element *quantization error*

$$\boldsymbol{\epsilon} \;=\; \widetilde{\mathbf{W}} - \mathbf{W}, \tag{1}$$

so that

$$\widetilde{\mathbf{y}} \;=\; \widetilde{\mathbf{W}}\,\mathbf{x} + \mathbf{b} \;=\; \mathbf{W}\,\mathbf{x} + \boldsymbol{\epsilon}\,\mathbf{x} + \mathbf{b} \;=\; \mathbf{y} + \boldsymbol{\epsilon}\,\mathbf{x}. \tag{2}$$

We assume Batch Normalization (BN) is folded into the preceding weight layer, but that the BN scale $\boldsymbol{\gamma}$ and shift $\boldsymbol{\beta}$ have been captured before fusion. The activation function is ReLU. Under the assumption made in [1] that the pre-ReLU output of a BN-folded layer is approximately Gaussian, the per-channel input distribution to a downstream layer follows a *ReLU-clipped normal* with parameters $(\boldsymbol{\beta}_c, \boldsymbol{\gamma}_c^2)$.

Throughout this document we use:

- c_o, c_i for output / input channel indices.
- m, n for kernel spatial indices.
- $\mathbb{E}[\mathbf{x}_{c_i}]$ and $\mathrm{Var}[\mathbf{x}_{c_i}]$ for the per-channel mean and variance of the layer input, assumed spatially stationary (i.i.d. across spatial positions within a channel).
- Independence *between* input channels (a standard analytical simplification; the same as [1]).

2 Bias Correction — Linear Layer

For a fully-connected layer [1, Eq. 16–17],

$$\mathbb{E}[\mathbf{y}] \;=\; \mathbb{E}[\mathbf{y}] + \mathbb{E}[\boldsymbol{\epsilon}\,\mathbf{x}] - \mathbb{E}[\boldsymbol{\epsilon}\,\mathbf{x}] \tag{3}$$

$$\;=\; \mathbb{E}[\widetilde{\mathbf{y}}] - \mathbb{E}[\boldsymbol{\epsilon}\,\mathbf{x}], \tag{4}$$

so subtracting $\mathbb{E}[\boldsymbol{\epsilon}\,\mathbf{x}] = \boldsymbol{\epsilon}\,\mathbb{E}[\mathbf{x}]$ from $\widetilde{\mathbf{y}}$ recovers the FP32 mean. Per output unit c_o,

$$\Delta b_{c_o} \;=\; \sum_{c_i} \boldsymbol{\epsilon}_{c_o,c_i}\,\mathbb{E}[\mathbf{x}_{c_i}]. \tag{5}$$

The correction is folded into the bias:

$$\mathbf{b}_{c_o}^{\mathrm{corr}} \;=\; \mathbf{b}_{c_o} - \Delta b_{c_o}. \tag{6}$$

3 Bias Correction — Convolutional Layer with ReLU

For a 2D convolution $\mathbf{y}_{c_o,i,j} = \sum_{c_i,m,n} \mathbf{W}_{c_o,c_i,m,n}\, \mathbf{x}_{c_i,i-m,j-n} + \mathbf{b}_{c_o}$, the same decomposition gives $\mathbb{E}[\boldsymbol{\epsilon} * \mathbf{x}]$ pointwise. Under spatial stationarity $\mathbb{E}[\mathbf{x}_{c_i,i-m,j-n}] = \mathbb{E}[\mathbf{x}_{c_i}]$:

$$\big[\boldsymbol{\epsilon} * \mathbb{E}[\mathbf{x}]\big]_{c_o,i,j} = \sum_{c_i,m,n} \mathbb{E}[\mathbf{x}_{c_i,i-m,j-n}]\, \boldsymbol{\epsilon}_{c_o,c_i,m,n} \tag{7}$$

$$= \sum_{c_i} \left[\mathbb{E}[\mathbf{x}_{c_i}] \sum_{m,n} \boldsymbol{\epsilon}_{c_o,c_i,m,n} \right]. \tag{8}$$

This is independent of the spatial location (i, j) and can be folded into the per-output-channel bias. In vector form, with $\boldsymbol{\epsilon}_\Sigma \in \mathbb{R}^{C_o \times C_i}$ defined by $[\boldsymbol{\epsilon}_\Sigma]_{c_o,c_i} = \sum_{m,n} \boldsymbol{\epsilon}_{c_o,c_i,m,n}$,

$$\boxed{\Delta\mathbf{b} \;=\; \boldsymbol{\epsilon}_\Sigma\, \mathbb{E}[\mathbf{x}], \qquad \mathbf{b}^{\,\mathrm{corr}} \;=\; \mathbf{b} - \Delta\mathbf{b}.} \tag{9}$$

Since (6) is the special case $K = 1$ of (9) (or equivalently $\boldsymbol{\epsilon}_\Sigma = \boldsymbol{\epsilon}$ for a fully-connected layer), the implementation handles both with a single contraction.

4 Variance Correction — Linear Layer

The paper of Nagel et al. corrects only the mean. However, weight quantization perturbs the per-channel variance as well. Under per-channel input independence,

$$\mathrm{Var}[\mathbf{y}_{c_o}] \;=\; \sum_{c_i} \mathbf{W}^2_{c_o,c_i}\, \mathrm{Var}[\mathbf{x}_{c_i}], \qquad \mathrm{Var}[\widetilde{\mathbf{y}}_{c_o}] \;=\; \sum_{c_i} \widetilde{\mathbf{W}}^2_{c_o,c_i}\, \mathrm{Var}[\mathbf{x}_{c_i}]. \tag{10}$$

Define the per-output-channel rescaling

$$s_{c_o} \;=\; \sqrt{\mathrm{Var}[\mathbf{y}_{c_o}] \,/\, \mathrm{Var}[\widetilde{\mathbf{y}}_{c_o}]}\,. \tag{11}$$

We apply a *mean-preserving* affine rescaling so that the second moment matches FP32 without disturbing whatever mean is in place:

$$\widetilde{\mathbf{y}}^{\,\mathrm{corr}}_{c_o} \;=\; s_{c_o}\big(\widetilde{\mathbf{y}}_{c_o} - \mathbb{E}[\widetilde{\mathbf{y}}_{c_o}]\big) + \mathbb{E}[\widetilde{\mathbf{y}}_{c_o}] \;=\; s_{c_o}\, \widetilde{\mathbf{y}}_{c_o} + (1 - s_{c_o})\, \mathbb{E}[\widetilde{\mathbf{y}}_{c_o}]. \tag{12}$$

Folding s_{c_o} into the per-channel weight row and into the bias gives:

$$\boxed{\widetilde{\mathbf{W}}^{\,\mathrm{corr}}_{c_o} \;=\; s_{c_o}\, \widetilde{\mathbf{W}}_{c_o}, \qquad \mathbf{b}^{\,\mathrm{corr}}_{c_o} \;=\; s_{c_o}\, \mathbf{b}_{c_o} + (1 - s_{c_o})\, \mathbb{E}[\widetilde{\mathbf{y}}_{c_o}].} \tag{13}$$

Here $\mathbb{E}[\widetilde{\mathbf{y}}_{c_o}] = \sum_{c_i} \widetilde{\mathbf{W}}_{c_o,c_i}\, \mathbb{E}[\mathbf{x}_{c_i}] + \mathbf{b}_{c_o}$ is the (uncorrected) quantized mean.

5 Variance Correction — Convolutional Layer with ReLU

Let $\widetilde{\mathbf{W}}^{(2)}_\Sigma \in \mathbb{R}^{C_o \times C_i}$ with $[\widetilde{\mathbf{W}}^{(2)}_\Sigma]_{c_o,c_i} = \sum_{m,n} \widetilde{\mathbf{W}}^2_{c_o,c_i,m,n}$, and similarly for $\mathbf{W}^{(2)}_\Sigma$. Under the same independence and spatial-i.i.d. assumptions used by [1, App. B],

$$\mathrm{Var}[\mathbf{y}_{c_o,i,j}] \;=\; \sum_{c_i} [\mathbf{W}^{(2)}_\Sigma]_{c_o,c_i}\, \mathrm{Var}[\mathbf{x}_{c_i}], \tag{14}$$

which is independent of the spatial location (i, j). Defining s_{c_o} as before, the same folding as (13) holds, with the convention that s_{c_o} broadcasts across the kernel spatial axes:

$$\boxed{\widetilde{\mathbf{W}}^{\,\mathrm{corr}}_{c_o,c_i,m,n} \;=\; s_{c_o}\, \widetilde{\mathbf{W}}_{c_o,c_i,m,n}, \qquad \mathbf{b}^{\,\mathrm{corr}}_{c_o} \;=\; s_{c_o}\, \mathbf{b}_{c_o} + (1 - s_{c_o})\, \mathbb{E}[\widetilde{\mathbf{y}}_{c_o}],} \tag{15}$$

with $\mathbb{E}[\widetilde{\mathbf{y}}_{c_o}] = \sum_{c_i} [\widetilde{\mathbf{W}}_\Sigma]_{c_o,c_i}\, \mathbb{E}[\mathbf{x}_{c_i}] + \mathbf{b}_{c_o}$.

Numerical guard. If $\mathrm{Var}[\widetilde{\mathbf{y}}_{c_o}]$ is near-zero (a near-dead channel), s_{c_o} explodes. We clip $s_{c_o} \in [0.5, 2.0]$ in the implementation.

6 Joint Bias & Variance Correction

When both corrections are applied, the variance scaling must pivot around the bias-corrected (FP32) mean rather than around the uncorrected quantized mean. Let

$$\mathbf{b}^{\,\mathrm{bias}} \;=\; \mathbf{b} - \Delta\mathbf{b} \quad \text{and} \quad \mathbb{E}[\mathbf{y}_{c_o}] \;=\; \sum_{c_i} [\mathbf{W}_\Sigma]_{c_o,c_i}\, \mathbb{E}[\mathbf{x}_{c_i}] + \mathbf{b}_{c_o}, \tag{16}$$

where $\Delta\mathbf{b}$ is from (9) and $\mathbb{E}[\mathbf{y}_{c_o}]$ is the FP32 target mean. The variance scaling is then applied around $\mathbb{E}[\mathbf{y}_{c_o}]$:

$$\boxed{\widetilde{\mathbf{W}}^{\,\mathrm{joint}}_{c_o} \;=\; s_{c_o}\, \widetilde{\mathbf{W}}_{c_o}, \qquad \mathbf{b}^{\,\mathrm{joint}}_{c_o} \;=\; s_{c_o}\big(\mathbf{b}_{c_o} - \Delta b_{c_o}\big) + (1 - s_{c_o})\, \mathbb{E}[\mathbf{y}_{c_o}].} \tag{17}$$

A direct check confirms that $\mathbb{E}[\mathbf{y}^{\,\mathrm{joint}}_{c_o}] = \mathbb{E}[\mathbf{y}_{c_o}]$ and $\mathrm{Var}[\mathbf{y}^{\,\mathrm{joint}}_{c_o}] = \mathrm{Var}[\mathbf{y}_{c_o}]$ under the working assumptions.

7 Computing $\mathbb{E}[\mathbf{x}]$ and $\mathrm{Var}[\mathbf{x}]$ from Batch-Norm Parameters

When BN is applied before ReLU, the pre-ReLU output of channel c is approximately $\mathcal{N}\big(\mu = \boldsymbol{\beta}_c,\, \sigma^2 = \boldsymbol{\gamma}_c^2\big)$. The post-ReLU input to the next layer is then the *clipped normal* on $[a, b] = [0, \infty)$. With $\alpha = -\boldsymbol{\beta}_c/\boldsymbol{\gamma}_c$:

$$\mathbb{E}[\mathbf{x}_c] \;=\; \boldsymbol{\gamma}_c\, \varphi(\alpha) + \boldsymbol{\beta}_c\big(1 - \Phi(\alpha)\big), \tag{18}$$

$$\mathrm{Var}[\mathbf{x}_c] \;=\; (\boldsymbol{\beta}_c^2 + \boldsymbol{\gamma}_c^2)\big(1 - \Phi(\alpha)\big) + \boldsymbol{\gamma}_c\boldsymbol{\beta}_c\, \varphi(\alpha) - \mathbb{E}[\mathbf{x}_c]^2, \tag{19}$$

where $\varphi(\cdot)$ is the standard normal PDF and $\Phi(\cdot)$ the standard normal CDF [1, App. C].

For the general clipped normal $f(X) = \mathrm{clip}(X, a, b)$ with $X \sim \mathcal{N}\big(\mu, \sigma^2\big)$ and $\alpha = (a - \mu)/\sigma$, $\beta_n = (b - \mu)/\sigma$, $Z = \Phi(\beta_n) - \Phi(\alpha)$, the closed-form mean is

$$\mathbb{E}[f(X)] = \sigma\big(\varphi(\alpha) - \varphi(\beta_n)\big) + \mu Z + a\Phi(\alpha) + b\big(1 - \Phi(\beta_n)\big), \tag{20}$$

and the variance follows from the second-moment formula in [1, App. C]. The implementation handles the general case with $a = 0$, $b = \infty$ as the ReLU specialization.

8 Propagating Statistics Through Residual Connections and GAP

Within a ResNet block, $\mathbb{E}[\mathbf{x}]$ and $\mathrm{Var}[\mathbf{x}]$ propagate as follows.

In-block conv inputs. For `conv2` (and `conv3` in a Bottleneck), the input is $\mathrm{ReLU}(\mathrm{BN}_{\text{prev conv}}(\cdot))$, so (18)–(19) apply with $(\boldsymbol{\beta}, \boldsymbol{\gamma})$ taken from the immediately preceding BN.

Residual sum. The output of the block is $\mathrm{ReLU}\big(\mathrm{BN}_{\mathrm{main}}(\cdot) + \mathrm{skip}\big)$. Treating skip as approximately Gaussian and independent of the main branch (a simplification used by [1]), the pre-ReLU per-channel distribution is

$$\mathcal{N}\big(\boldsymbol{\beta}_{\mathrm{main}} + \mu_{\mathrm{skip}},\; \boldsymbol{\gamma}^2_{\mathrm{main}} + \sigma^2_{\mathrm{skip}}\big)\,, \tag{21}$$

and the post-ReLU activation statistics follow from (18)–(19).

- *Identity skip:* $(\mu_{\mathrm{skip}}, \sigma^2_{\mathrm{skip}})$ is the previous block's post-residual stats (recursive).
- *Downsample skip:* the skip path is also $\mathrm{BN}_{\mathrm{ds}}$, contributing $(\boldsymbol{\beta}_{\mathrm{ds}}, \boldsymbol{\gamma}^2_{\mathrm{ds}})$.

Global average pooling (`fc` input). The fully-connected layer of a ResNet sees $\bar{\mathbf{x}}_c = \frac{1}{HW}\sum_{i,j}\mathbf{x}_{c,i,j}$. Under the spatial-i.i.d. assumption,

$$\mathbb{E}[\bar{\mathbf{x}}_c] \;=\; \mathbb{E}[\mathbf{x}_c], \qquad \mathrm{Var}[\bar{\mathbf{x}}_c] \;=\; \mathrm{Var}[\mathbf{x}_c]\,/\,(HW), \tag{22}$$

so the FC layer uses the layer-4 output mean unchanged and a variance scaled by $1/(HW)$ (e.g. $1/49$ for a 7×7 feature map at 224×224 input).

Stem. The first convolution receives the normalized image, not the output of a BN/ReLU. Analytical correction is undefined here and the stem layer is left uncorrected in the implementation.

9 Algorithm

Algorithm 1: Analytical bias / variance correction for a quantized model.

Input: FP32 model with BN; quantization method; correction mode $\in$ {none, bias, variance, joint}.
Output: Corrected quantized model.
$\{(\boldsymbol{\gamma}_b, \boldsymbol{\beta}_b)\}_b \leftarrow$ `capture_bn_params(model)`;
fold BN into Conv (in place);
$\{(\mathbb{E}[\mathbf{x}], \mathrm{Var}[\mathbf{x}])\}_L \leftarrow$ `compute_input_stats`, propagating per §8;
for *each weight layer L except stem* **do**
  Save $\mathbf{W}_L^{\mathrm{fp}}$;
  Quantize: $\widetilde{\mathbf{W}}_L$;
  $\boldsymbol{\epsilon}_\Sigma \leftarrow (\widetilde{\mathbf{W}} - \mathbf{W}^{\mathrm{fp}})$.sum(kernel axes);
  $\Delta\mathbf{b} \leftarrow \boldsymbol{\epsilon}_\Sigma \cdot \mathbb{E}[\mathbf{x}]_L$;
  $\mathbf{s} \leftarrow \sqrt{(\mathbf{W}_\Sigma^{\mathrm{fp}\,(2)}\,\mathrm{Var}[\mathbf{x}])\,/\,(\widetilde{\mathbf{W}}_\Sigma^{(2)}\,\mathrm{Var}[\mathbf{x}])}$, clipped;
  if *mode* = none **then**
    no change
  else if *mode* = bias **then**
    $\mathbf{b} \leftarrow \mathbf{b} - \Delta\mathbf{b}$
  else if *mode* = variance **then**
    $\mathbb{E}[\widetilde{\mathbf{y}}] \leftarrow \widetilde{\mathbf{W}}_\Sigma \cdot \mathbb{E}[\mathbf{x}] + \mathbf{b}$;
    $\widetilde{\mathbf{W}} \leftarrow \mathbf{s} \odot \widetilde{\mathbf{W}}$;;
    $\mathbf{b} \leftarrow \mathbf{s} \odot \mathbf{b} + (\mathbf{1} - \mathbf{s}) \odot \mathbb{E}[\widetilde{\mathbf{y}}]$;
  else *joint*
    $\mathbb{E}[\mathbf{y}] \leftarrow \mathbf{W}_\Sigma^{\mathrm{fp}} \cdot \mathbb{E}[\mathbf{x}] + \mathbf{b}$;
    $\widetilde{\mathbf{W}} \leftarrow \mathbf{s} \odot \widetilde{\mathbf{W}}$;;
    $\mathbf{b} \leftarrow \mathbf{s} \odot (\mathbf{b} - \Delta\mathbf{b}) + (\mathbf{1} - \mathbf{s}) \odot \mathbb{E}[\mathbf{y}]$;
  end
end

10 Implementation Notes

- **Variance and joint correction require per-channel weight quantization.** The scale s_{c_o} is per output channel by construction. Per-channel weight quantization stores one scale per output channel, so s_{c_o} folds cleanly into that scale (or, equivalently in this dequantized-FP32 pipeline, into the per-channel weight row). Per-tensor weight quantization stores a single shared scale; multiplying by a per-channel s_{c_o} produces a per-channel grid that is

no longer representable under per-tensor quant. To keep the benchmark over hardware-realizable variants only, the implementation restricts {bias, variance, joint} corrections to per-channel quantization variants. Per-tensor variants run only as the uncorrected baseline. (A hardware-equivalent alternative would be to express s_{c_o} as a per-output-channel rescale at the accumulator/requant boundary, leaving the integer weight tensor untouched; that is supported by typical INT8 inference engines but is out of scope here.)

- Bias correction is hardware-friendly in any quantization scheme: it is an additive shift on the bias tensor, which is FP32/INT32 in any standard pipeline.
- The independence assumption between input channels is the same simplification used by [1] for $\mathbb{E}[\mathbf{x}]$. It is the dominant approximation; real activations exhibit modest cross-channel correlation, which would add cross terms to $\mathrm{Var}[\mathbf{y}_{c_o}]$ that this derivation omits.
- The analytical clipped-normal model uses the BN running statistics. After fusion, the BN parameters $(\boldsymbol{\gamma}, \boldsymbol{\beta})$ are captured beforehand; the running mean and variance are absorbed into the fused weight and bias.
- The stem (first) convolution receives the normalized image and is left uncorrected: there is no preceding BN/ReLU to give an analytical input distribution.

References

[1] Markus Nagel, Mart van Baalen, Tijmen Blankevoort, Max Welling. *Data-Free Quantization Through Weight Equalization and Bias Correction.* ICCV 2019. `https://arxiv.org/abs/1906.04721`